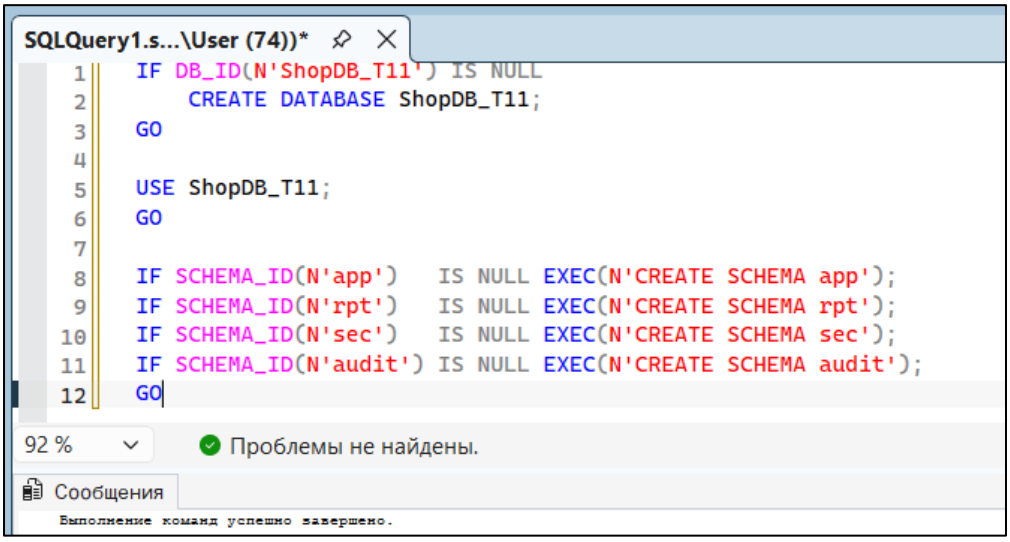
Билет 11: «Locks и blocking»

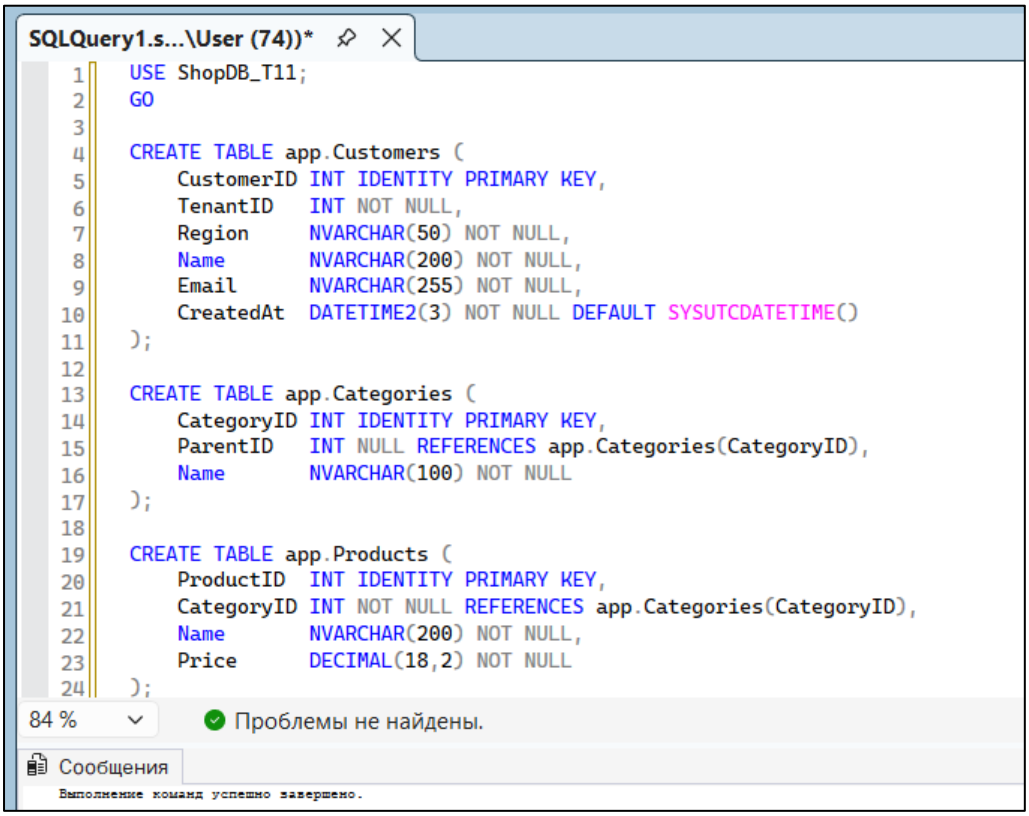
Подготовка БД ShopDB_T11

Создаем БД и схемы



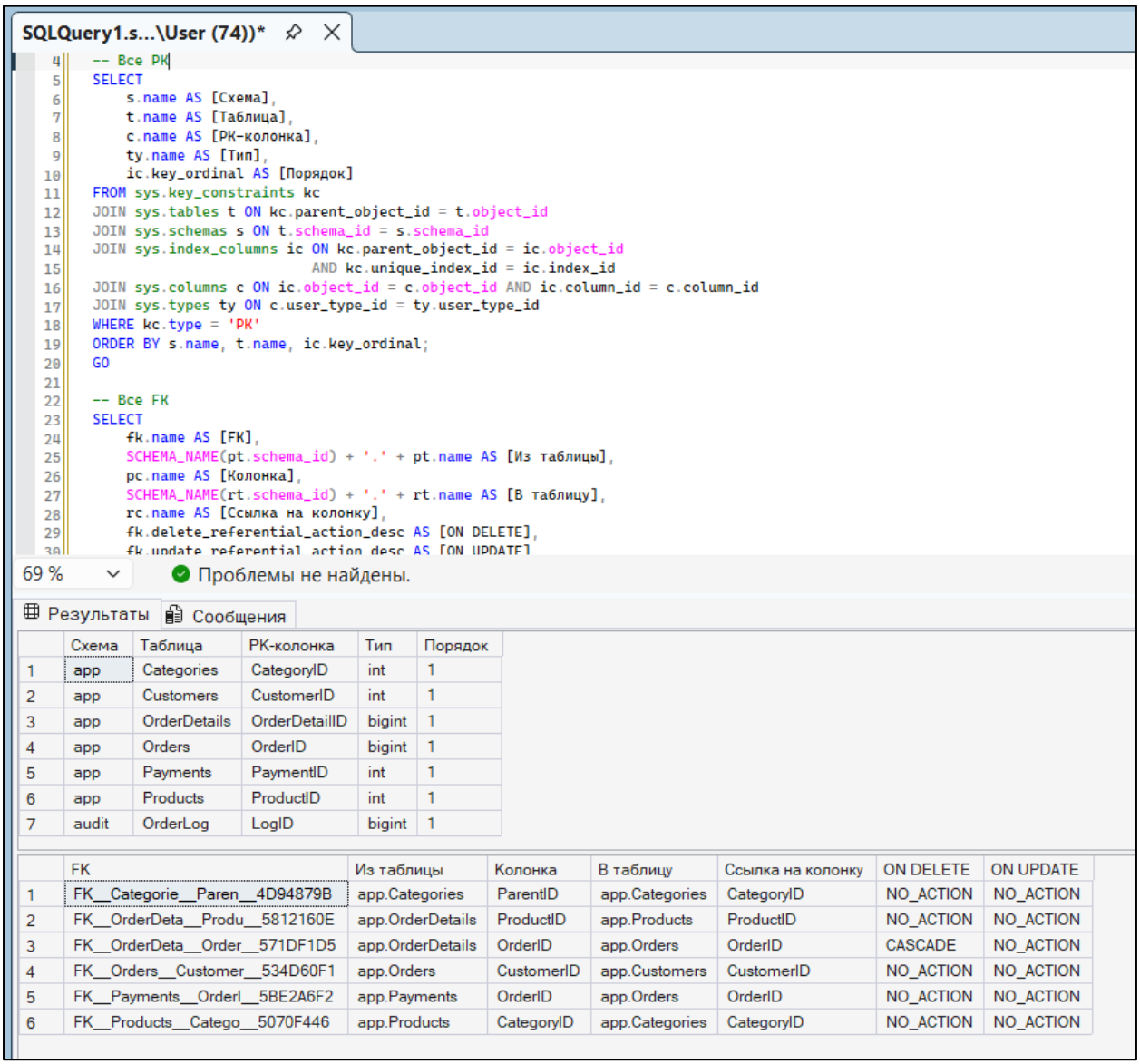
БД ShopDB_T11 создана, схемы app/rpt/sec/audit добавлены.

Создание таблицы



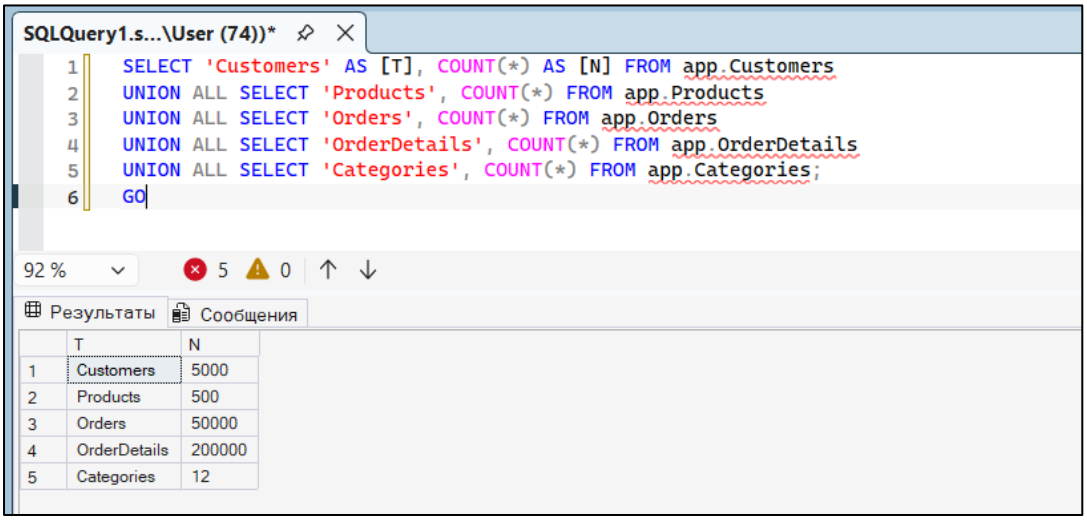
Создано 7 таблиц: 6 в app + audit.OrderLog.

PK/FK — анализ



Анализ PK/FK: 7 PK, 6 FK

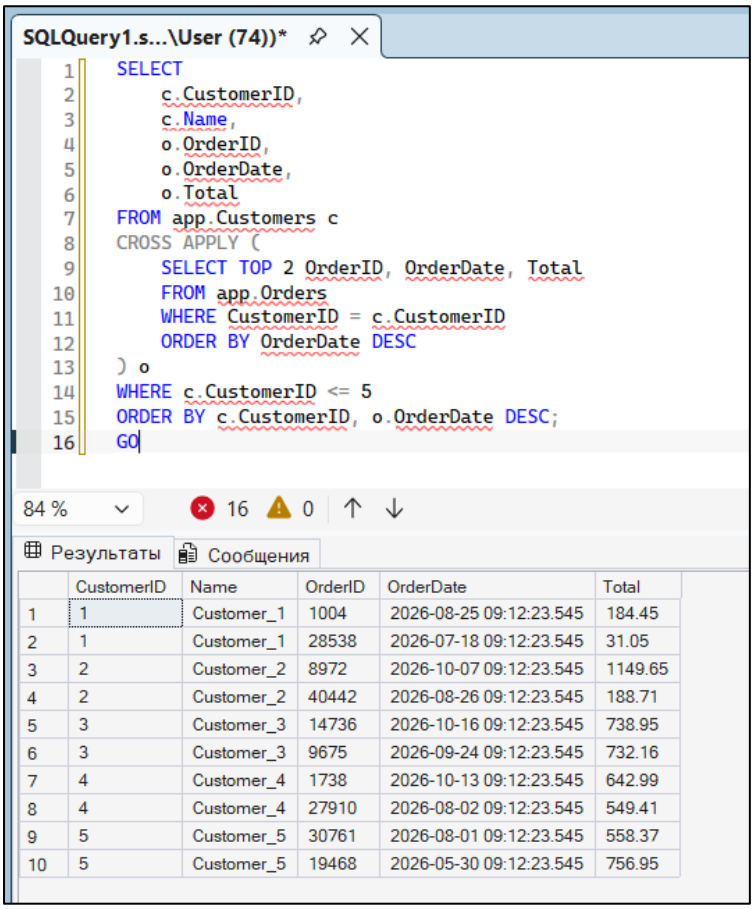
Наполнение данными



БД наполнена: 5000 клиентов, 500 товаров, 50000 заказов, 200000 позиций, 12 категорий.

ЗАДАНИЕ 2. Оконные функции + APPLY + VIEW/процедуры/функции

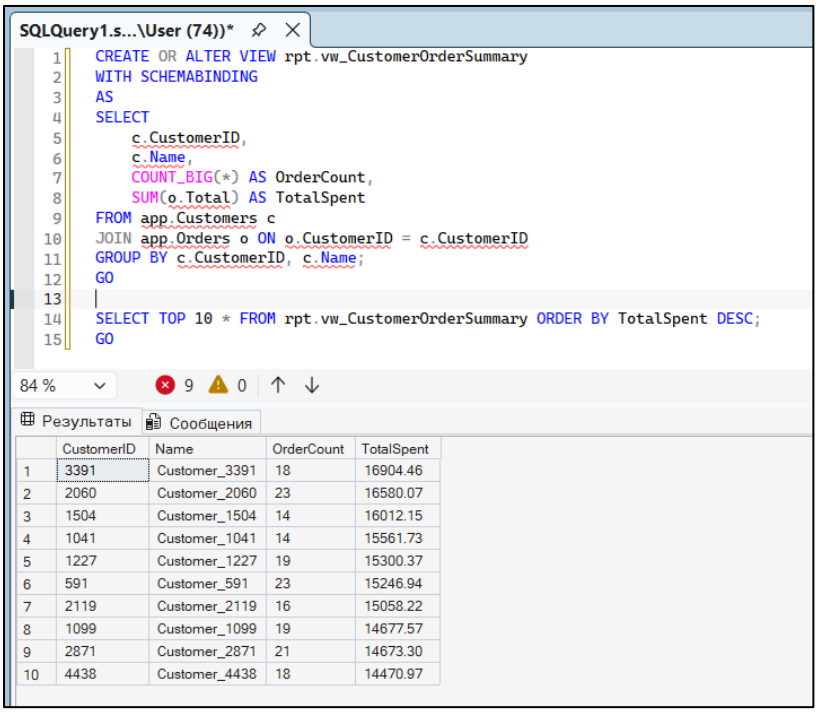
ROW_NUMBER + SUM OVER (накопительный итог)



	CustomerID	Name	OrderID	OrderDate	Total
1	1	Customer_1	1004	2026-08-25 09:12:23.545	184.45
2	1	Customer_1	28538	2026-07-18 09:12:23.545	31.05
3	2	Customer_2	8972	2026-10-07 09:12:23.545	1149.65
4	2	Customer_2	40442	2026-08-26 09:12:23.545	188.71
5	3	Customer_3	14736	2026-10-16 09:12:23.545	738.95
6	3	Customer_3	9675	2026-09-24 09:12:23.545	732.16
7	4	Customer_4	1738	2026-10-13 09:12:23.545	642.99
8	4	Customer_4	27910	2026-08-02 09:12:23.545	549.41
9	5	Customer_5	30761	2026-08-01 09:12:23.545	558.37
10	5	Customer_5	19468	2026-05-30 09:12:23.545	756.95

CROSS APPLY: 2 последних заказа каждого клиента.

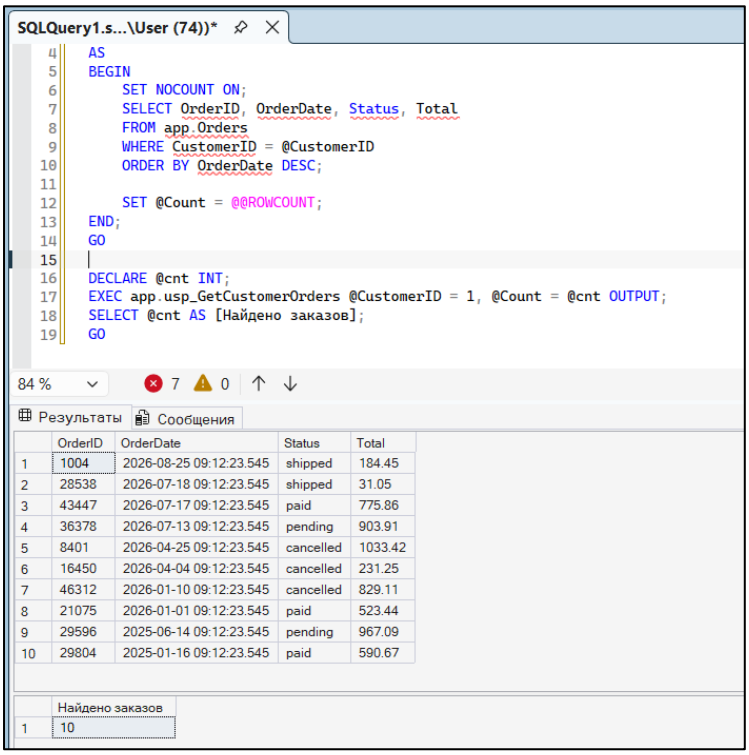
VIEW со SCHEMABINDING



	CustomerID	Name	OrderCount	TotalSpent
1	3391	Customer_3391	18	16904.46
2	2060	Customer_2060	23	16580.07
3	1504	Customer_1504	14	16012.15
4	1041	Customer_1041	14	15561.73
5	1227	Customer_1227	19	15300.37
6	591	Customer_591	23	15246.94
7	2119	Customer_2119	16	15058.22
8	1099	Customer_1099	19	14677.57
9	2871	Customer_2871	21	14673.30
10	4438	Customer_4438	18	14470.97

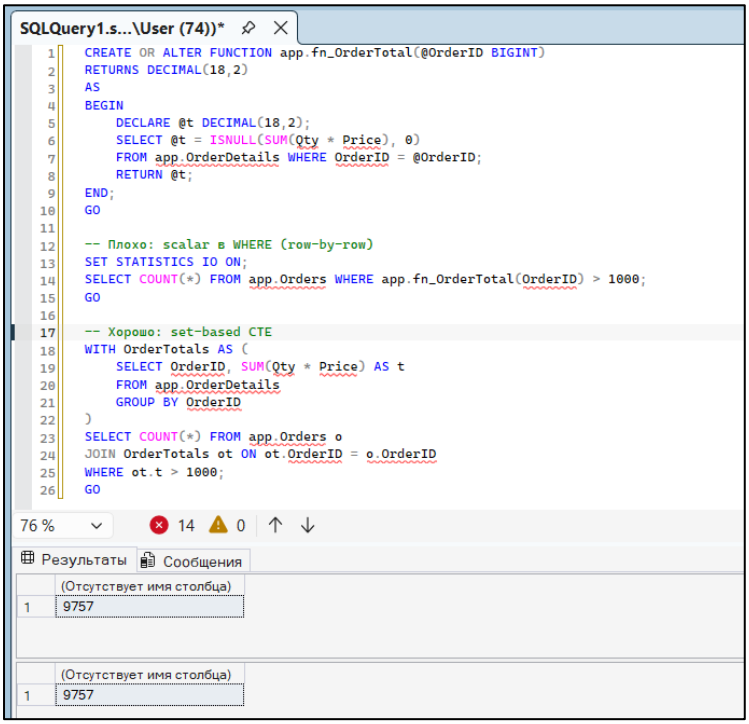
VIEW rpt.vw_CustomerOrderSummary со SCHEMABINDING.

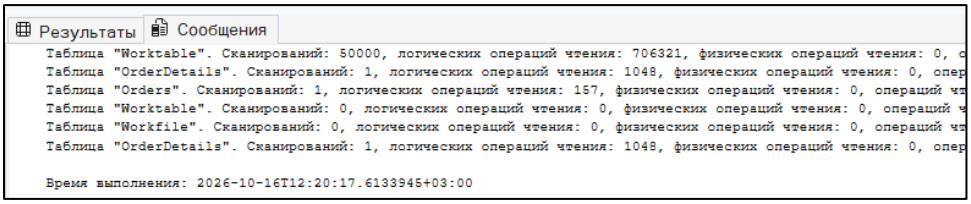
Процедура с OUTPUT



Процедура с OUTPUT-параметром @Count.

Scalar-функция + проблема в WHERE





Scalar в WHERE = row-by-row (много reads). CTE = set-based (в разы меньше).

ЗАДАНИЕ 3. Триггер + транзакция + изоляция + deadlock

3.1. Триггер app.trg_OrderLog (multi-row)

SQLQuery1.s...\User (74))*

```
USE ShopDB_T11;
GO

CREATE OR ALTER TRIGGER app.trg_OrderLog
ON app.Orders
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO audit.OrderLog(OrderID, Action)
    SELECT OrderID, 'INSERT' FROM inserted
    WHERE NOT EXISTS (SELECT 1 FROM deleted d WHERE d.OrderID = inserted.OrderID);

    INSERT INTO audit.OrderLog(OrderID, Action)
    SELECT OrderID, 'UPDATE' FROM inserted
    WHERE EXISTS (SELECT 1 FROM deleted d WHERE d.OrderID = inserted.OrderID);

    INSERT INTO audit.OrderLog(OrderID, Action)
    SELECT OrderID, 'DELETE' FROM deleted
    WHERE NOT EXISTS (SELECT 1 FROM inserted i WHERE i.OrderID = deleted.OrderID);
END;
GO

INSERT INTO app.Orders(CustomerID, OrderDate, Status, Total)
VALUES (1, SYSUTCDATETIME(), 'pending', 100),
       (1, SYSUTCDATETIME(), 'paid', 200),
       (2, SYSUTCDATETIME(), 'shipped', 300);
GO

SELECT TOP 5 * FROM audit.OrderLog ORDER BY LogID DESC;
GO
```

76 % 26 0

Результаты Сообщения

	LogID	OrderID	Action	ChangedAt	ChangedBy
1	2	50001	INSERT	2026-10-16 09:24:36.767	TECNO-GUSEYN\User
2	1	50002	INSERT	2026-10-16 09:24:36.767	TECNO-GUSEYN\User
3	0	50003	INSERT	2026-10-16 09:24:36.767	TECNO-GUSEYN\User

Триггер AFTER INSERT multi-row: 3 строки - 3 записи в лог.

3.2. Транзакция + TRY/CATCH + XACT_STATE

SQLQuery1.s...\User (74))*

```
USE ShopDB_T11;
GO

BEGIN TRY
    BEGIN TRAN;
    UPDATE app.Orders SET Status = 'paid' WHERE OrderID = 1;
    INSERT INTO audit.OrderLog(OrderID, Action) VALUES (1, 'PAY');
    COMMIT;
    PRINT 'OK';
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0 ROLLBACK;
    SELECT ERROR_NUMBER() AS ErrNum, ERROR_MESSAGE() AS ErrMsg;
END CATCH;
GO
```

101 % 6 0

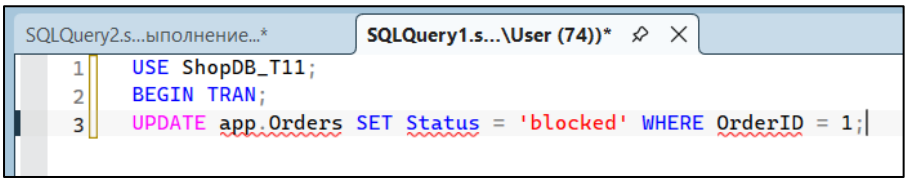
Сообщения

```
Таблица "Orders". Сканирований: 0, логических операций чтения: 2, физических операций чтения: 0, операций чт
Таблица "OrderLog". Сканирований: 0, логических операций чтения: 0, физических операций чтения: 0, операций
Таблица "OrderLog". Сканирований: 0, логических операций чтения: 2, физических операций чтения: 0, операций
Таблица "OrderLog". Сканирований: 0, логических операций чтения: 0, физических операций чтения: 0, операций
Таблица "OrderLog". Сканирований: 0, логических операций чтения: 2, физических операций чтения: 0, операций
OK
```

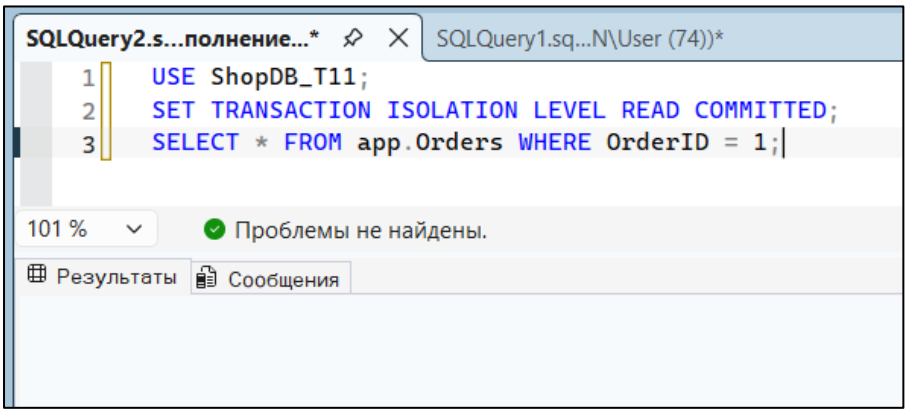
Транзакция с TRY/CATCH/XACT_STATE — успешный коммит.

3.3. Блокировка: 2 окна

Сессия A (окно 1)



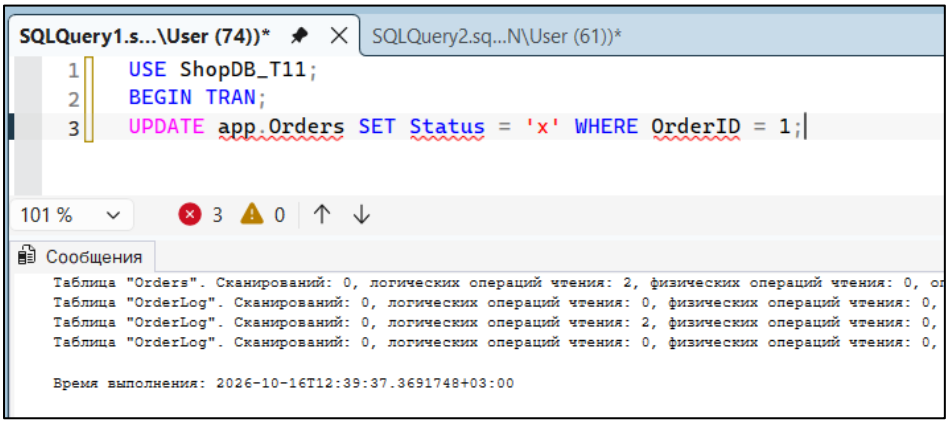
Сессия B (окно 2)



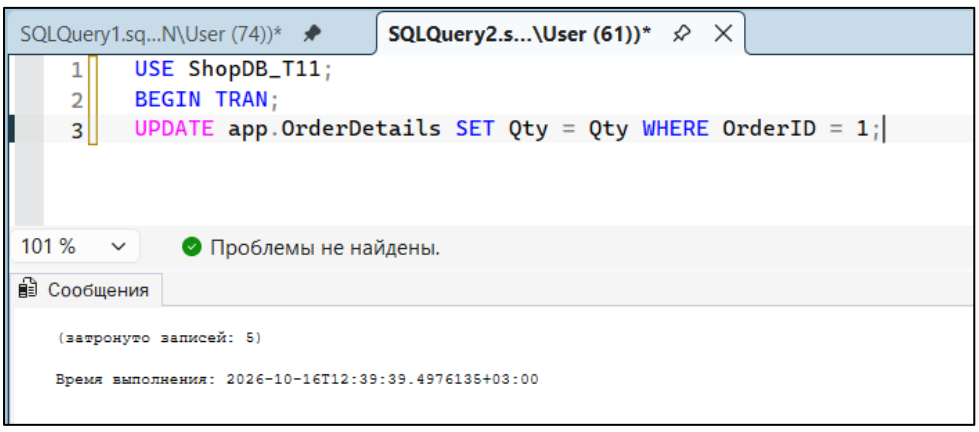
Сессия B зависла — ждёт снятия блокировки с OrderID = 1.

3.5. Deadlock

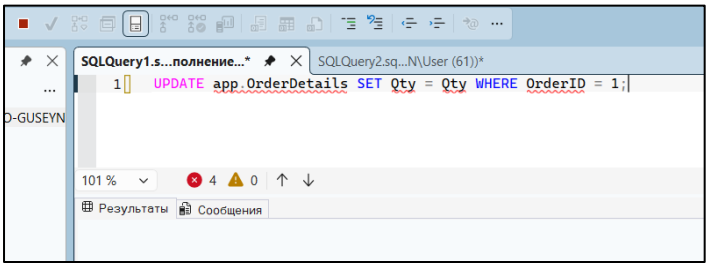
Окно 1



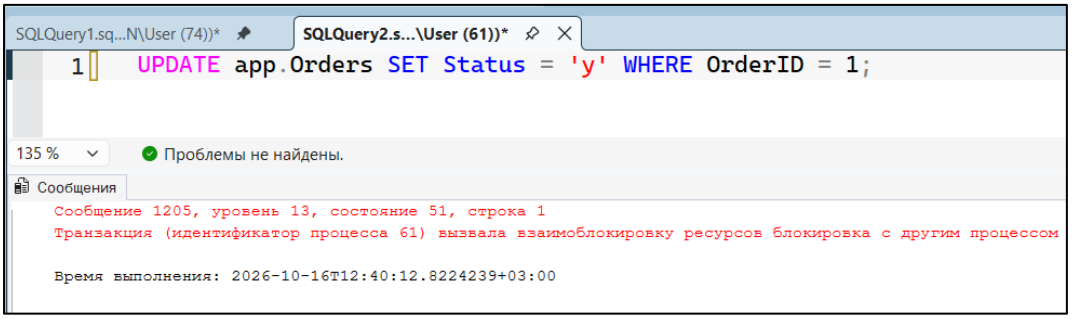
Окно 2



Окно 1



Окно 2

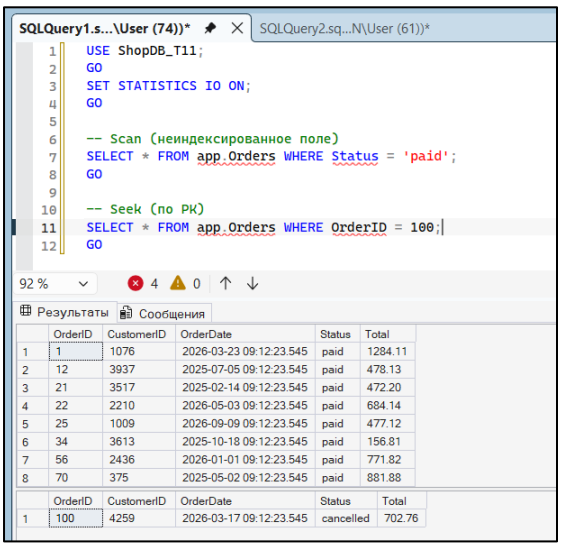


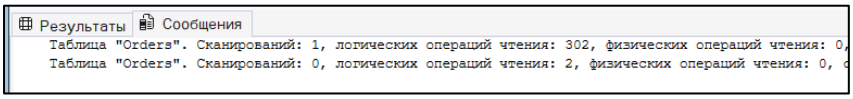
1. Сессия A держит блокировку на Orders.
2. Сессия B держит блокировку на OrderDetails.
3. Сессия A пытается взять OrderDetails - зависает (ждет B).
4. Сессия B пытается взять Orders - возникает цикл ожидания.

Результат: SQL Server обнаружил deadlock и выбрал жертвой SPID 61 (сессия B). Её транзакция откачена с ошибкой **1205**.

ЗАДАНИЕ 4. Планы + индексы + SARGability + Query Store + tempdb

4.1. Сравнение Scan и Seek (план + STATISTICS IO)





Scan (Status) — 302 reads. Seek (OrderID) — 2 reads. Разница в 150 раз.

4.2. Создание индексов

```sql
USE ShopDB_T11;
GO

-- Composite
CREATE NONCLUSTERED INDEX IX_Orders_CustDate
ON app.Orders(CustomerID, OrderDate);
GO

-- Covering (INCLUDE)
CREATE NONCLUSTERED INDEX IX_Orders_CustDate_INC
ON app.Orders(CustomerID, OrderDate)
INCLUDE (Status, Total);
GO

-- Filtered
CREATE NONCLUSTERED INDEX IX_Orders_Paid
ON app.Orders(OrderDate)
WHERE Status = 'paid';
GO

EXEC sp_helpindex 'app.Orders';
GO
```

Результаты | Сообщения

	index_name	index_description	index_keys
1	IX_Orders_CustDate	nonclustered located on PRIMARY	CustomerID, OrderDate
2	IX_Orders_CustDate_INC	nonclustered located on PRIMARY	CustomerID, OrderDate
3	IX_Orders_CustomerDate	nonclustered located on PRIMARY	CustomerID, OrderDate
4	IX_Orders_Paid	nonclustered located on PRIMARY	OrderDate
5	PK__Orders__C3905BAF65ED303E	clustered, unique, primary key located on PRIMARY	OrderID

Созданы: composite, covering (INCLUDE), filtered. Плюс PK clustered.

4.3. SARGability

```sql
USE ShopDB_T11;
GO
SET STATISTICS IO ON;
GO

--  NON-SARGable
SELECT OrderID FROM app.Orders WHERE YEAR(OrderDate) = 2025;
GO

--  SARGable (узкий диапазон)
SELECT OrderID FROM app.Orders
WHERE OrderDate >= '2025-01-01' AND OrderDate < '2025-02-01';
GO
```

Результаты | Сообщения

	OrderID
1	791
2	1515
3	2332
4	2359
5	2461

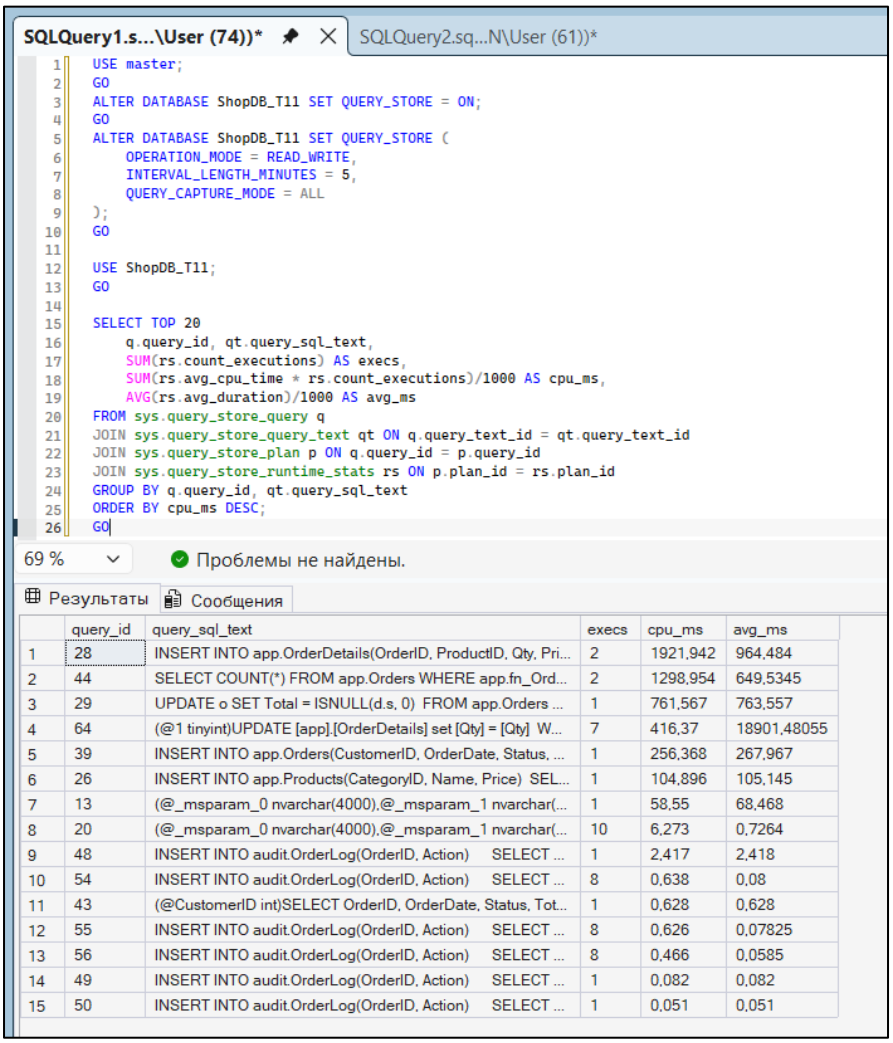
	OrderID
1	791
2	1515
3	2332
4	2359
5	2461

Результаты | Сообщения

```
Таблица "Orders". Сканирований: 1, логических операций чтения: 132, физических операций чтения: 0
Таблица "Orders". Сканирований: 1, логических операций чтения: 8, физических операций чтения: 0,
```

NON-SARGable YEAR(OrderDate)=2025 - *Scan (132 reads). SARGable диапазон - Seek (8 reads). Разница в 16 раз.*

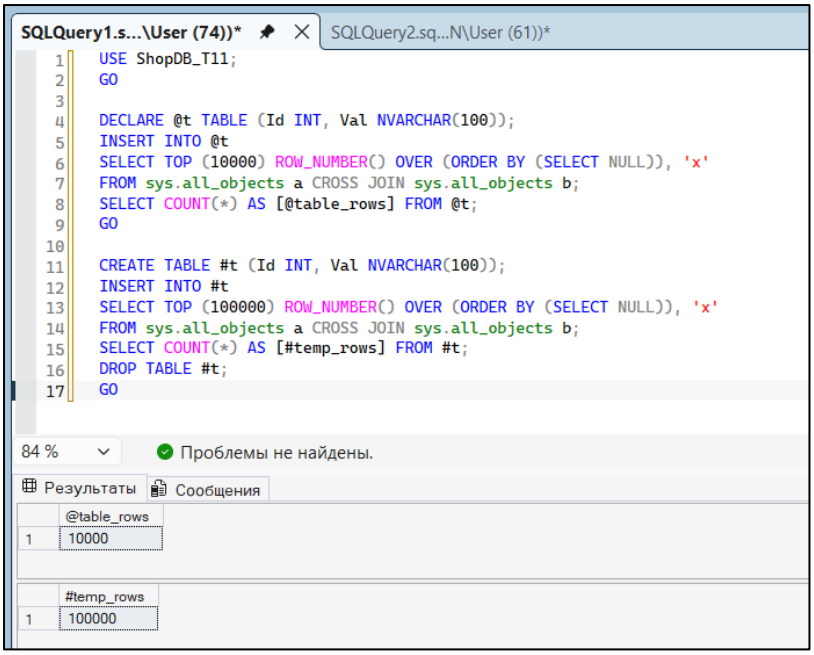
4.4. Query Store



	query_id	query_sql_text	execs	cpu_ms	avg_ms
1	28	INSERT INTO app.OrderDetails(OrderID, ProductID, Qty, Pri...	2	1921,942	964,484
2	44	SELECT COUNT(*) FROM app.Orders WHERE app.fn_Ord...	2	1298,954	649,5345
3	29	UPDATE o SET Total = ISNULL(d.s, 0) FROM app.Orders ...	1	761,567	763,557
4	64	(@1 tinyint)UPDATE [app].[OrderDetails] set [Qty] = [Qty] W...	7	416,37	18901,48055
5	39	INSERT INTO app.Orders(CustomerID, OrderDate, Status, ...	1	256,368	267,967
6	26	INSERT INTO app.Products(CategoryID, Name, Price) SEL...	1	104,896	105,145
7	13	(@_msparam_0 nvarchar(4000),@_msparam_1 nvarchar(...	1	58,55	68,468
8	20	(@_msparam_0 nvarchar(4000),@_msparam_1 nvarchar(...	10	6,273	0,7264
9	48	INSERT INTO audit.OrderLog(OrderID, Action) SELECT ...	1	2,417	2,418
10	54	INSERT INTO audit.OrderLog(OrderID, Action) SELECT ...	8	0,638	0,08
11	43	(@CustomerID int)SELECT OrderID, OrderDate, Status, Tot...	1	0,628	0,628
12	55	INSERT INTO audit.OrderLog(OrderID, Action) SELECT ...	8	0,626	0,07825
13	56	INSERT INTO audit.OrderLog(OrderID, Action) SELECT ...	8	0,466	0,0585
14	49	INSERT INTO audit.OrderLog(OrderID, Action) SELECT ...	1	0,082	0,082
15	50	INSERT INTO audit.OrderLog(OrderID, Action) SELECT ...	1	0,051	0,051

Query Store включён, топ-20 запросов по CPU.

4.5. #temp vs @table



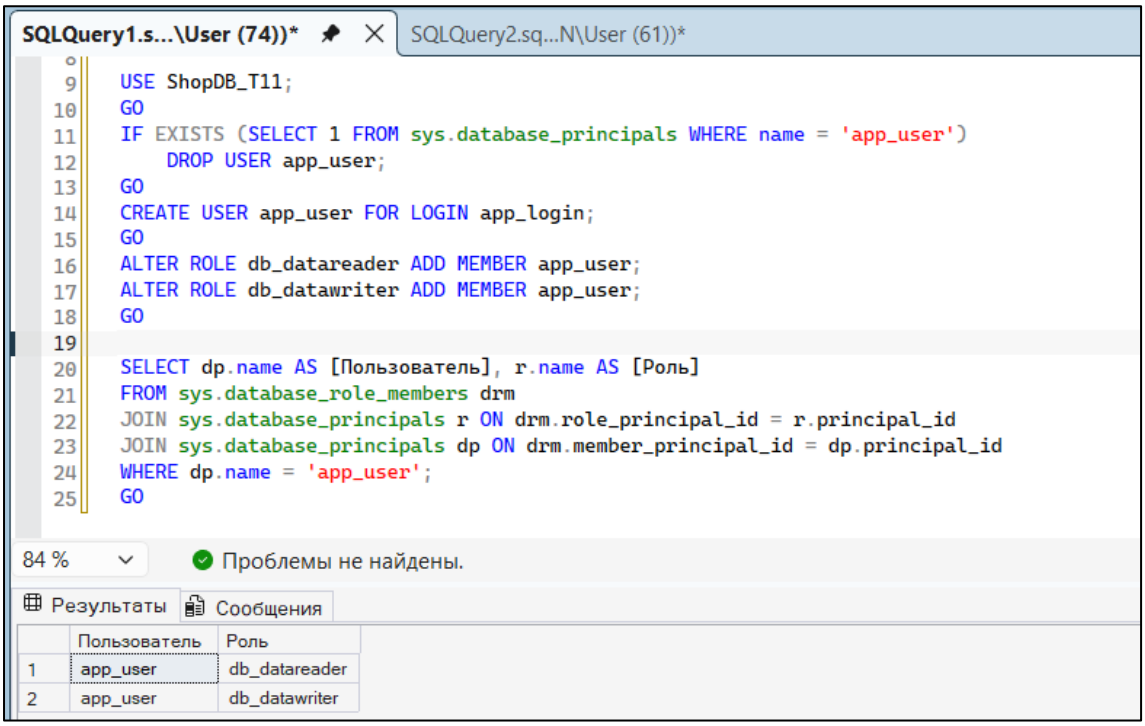
	@table_rows
1	10000

	#temp_rows
1	100000

@table — в памяти до ~10k строк. #temp — в tempdb, работает с 100k+, поддерживает индексы.

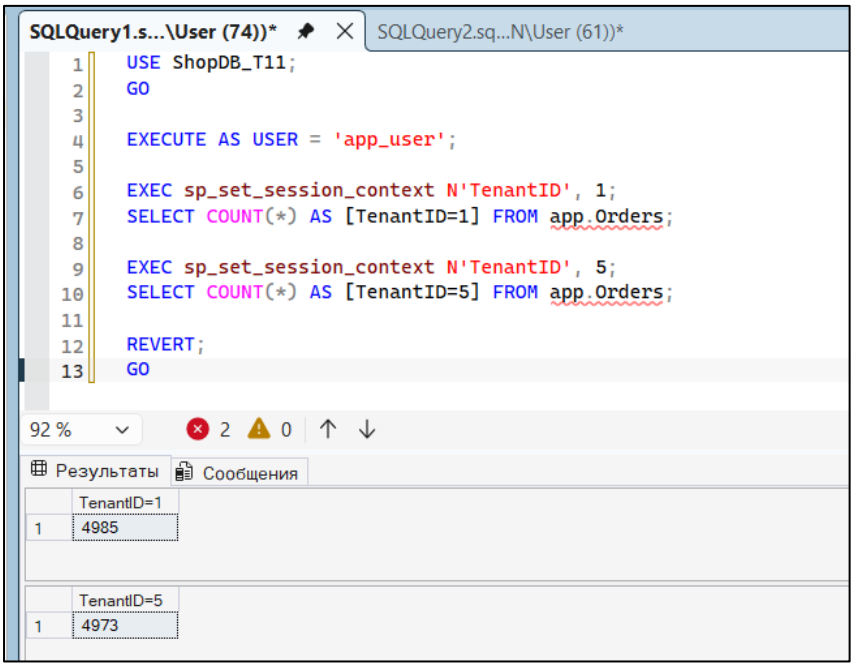
Задание 5. Безопасность + RLS + аудит + backup

5.1. Логин + Пользователь + Роли



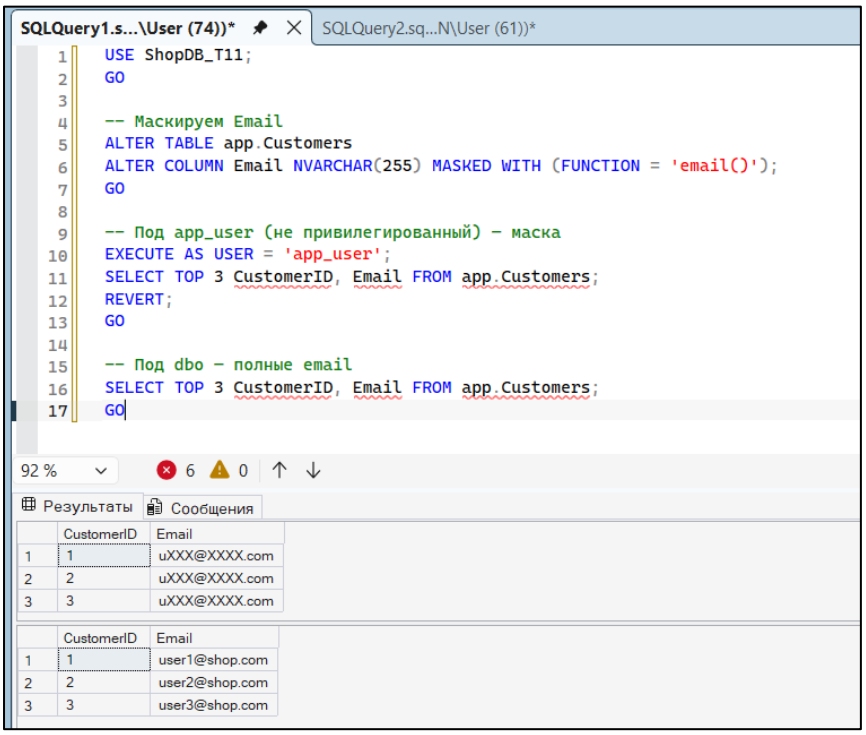
Логин app_login + user app_user, добавлен в роли db_datareader и db_datawriter.

5.2. RLS по TenantID



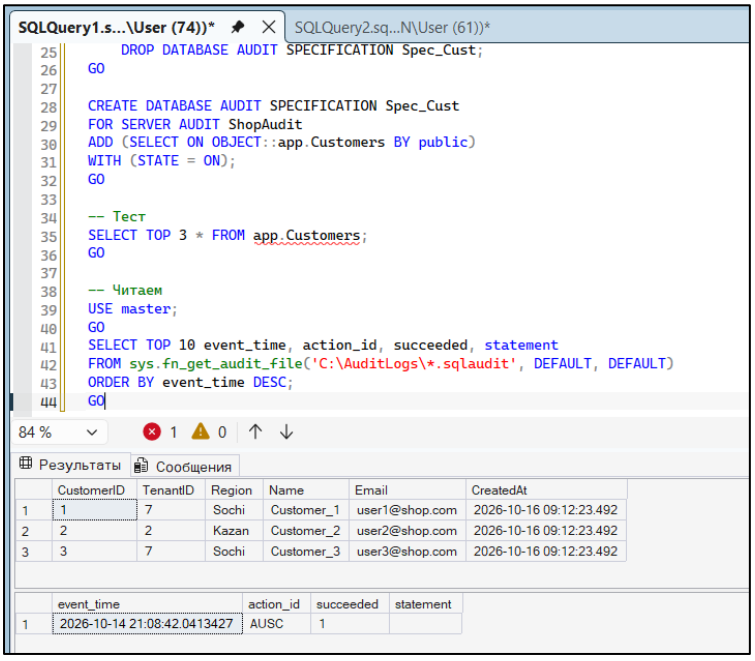
RLS по TenantID под app_user: TenantID=1 и TenantID=5 видят разные строки. db_owner видит все — это by design.

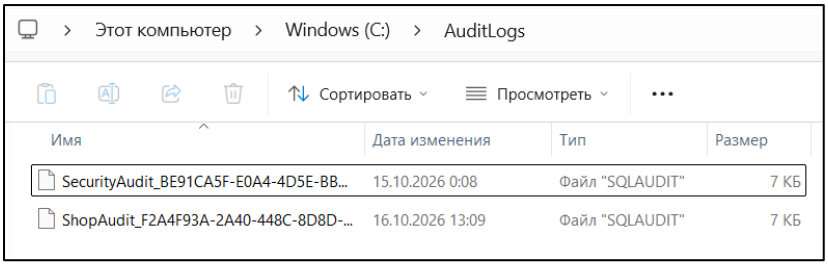
5.3. Dynamic Data Masking



DDM: Email маскируется для app_user, dbo видит полные данные (UNMASK).

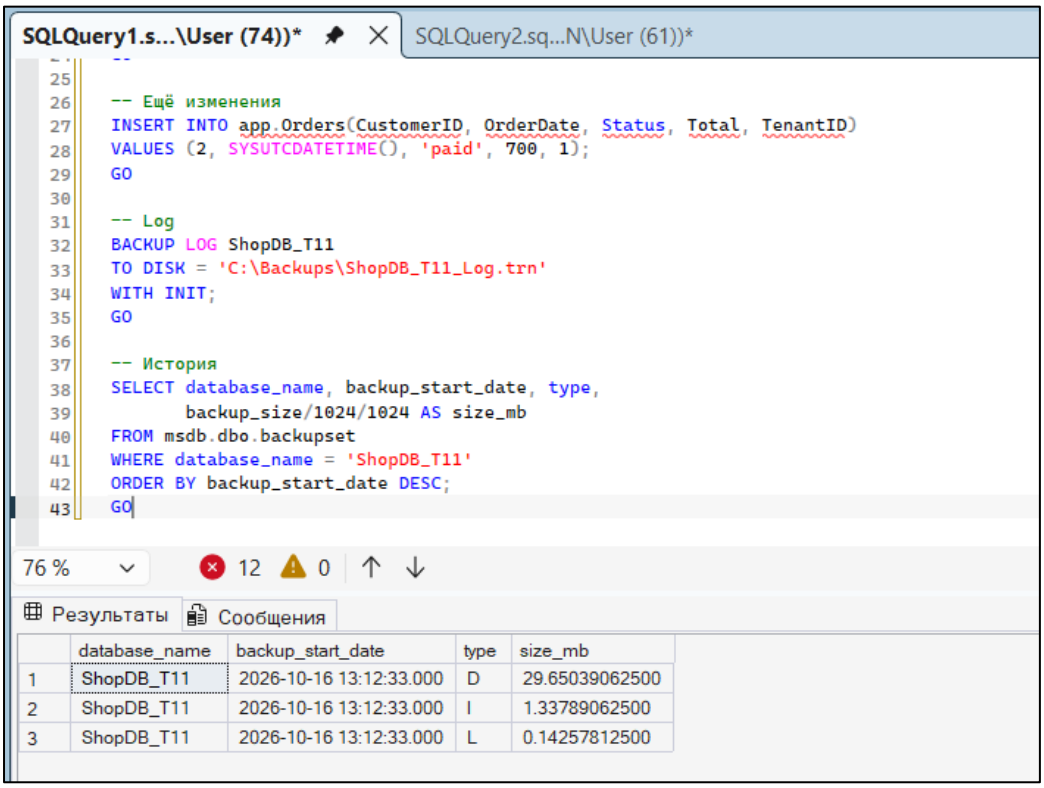
5.4. Server Audit

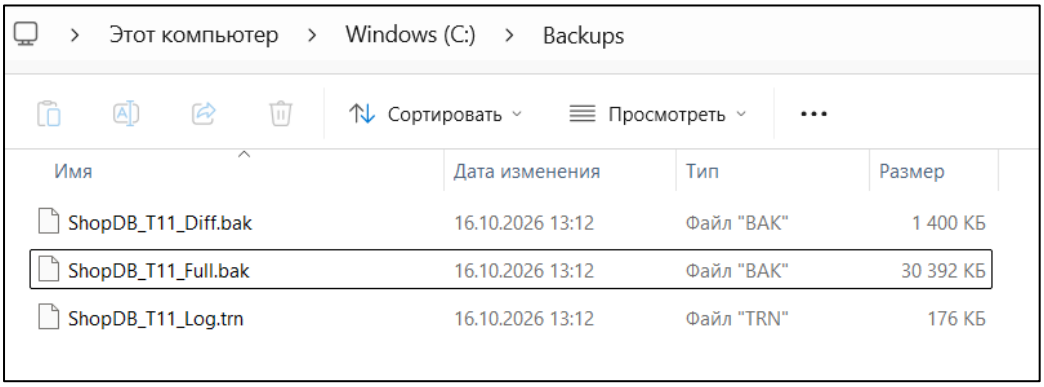




Server Audit ShopAudit: SELECT к app.Customers зафиксирован в логе.

5.5. Backup chain (Full → Diff → Log)





Backup chain: Full - Diff - Log. (Express Edition не поддерживает COMPRESSION.)